



Bildquelle:

<https://de.wikipedia.org/wiki/Java-Technologie>

1 Einführung in Java

1.1 Java SE

1.2 Typsystem

1.3 Anweisungen

1.4 Klassenmethoden

1.5 Felder



Bildquelle:

<https://de.wikipedia.org/wiki/Java-Technologie>

1 Einführung in Java

1.1 Java SE

1.2 Typsystem

1.3 Anweisungen

1.4 Klassenmethoden

1.5 Felder

Java = Programmiersprache + Java Virtual Machine + Bibliothek

- weit verbreitet (Millionen von Entwickler*innen, Milliarden von Geräten)
- allgemein verfügbar (Linux, macOS, Windows)
- seit Mitte der 1990er Jahre in ständiger Entwicklung
- bietet moderne Programmierkonzepte u. -abstraktionen

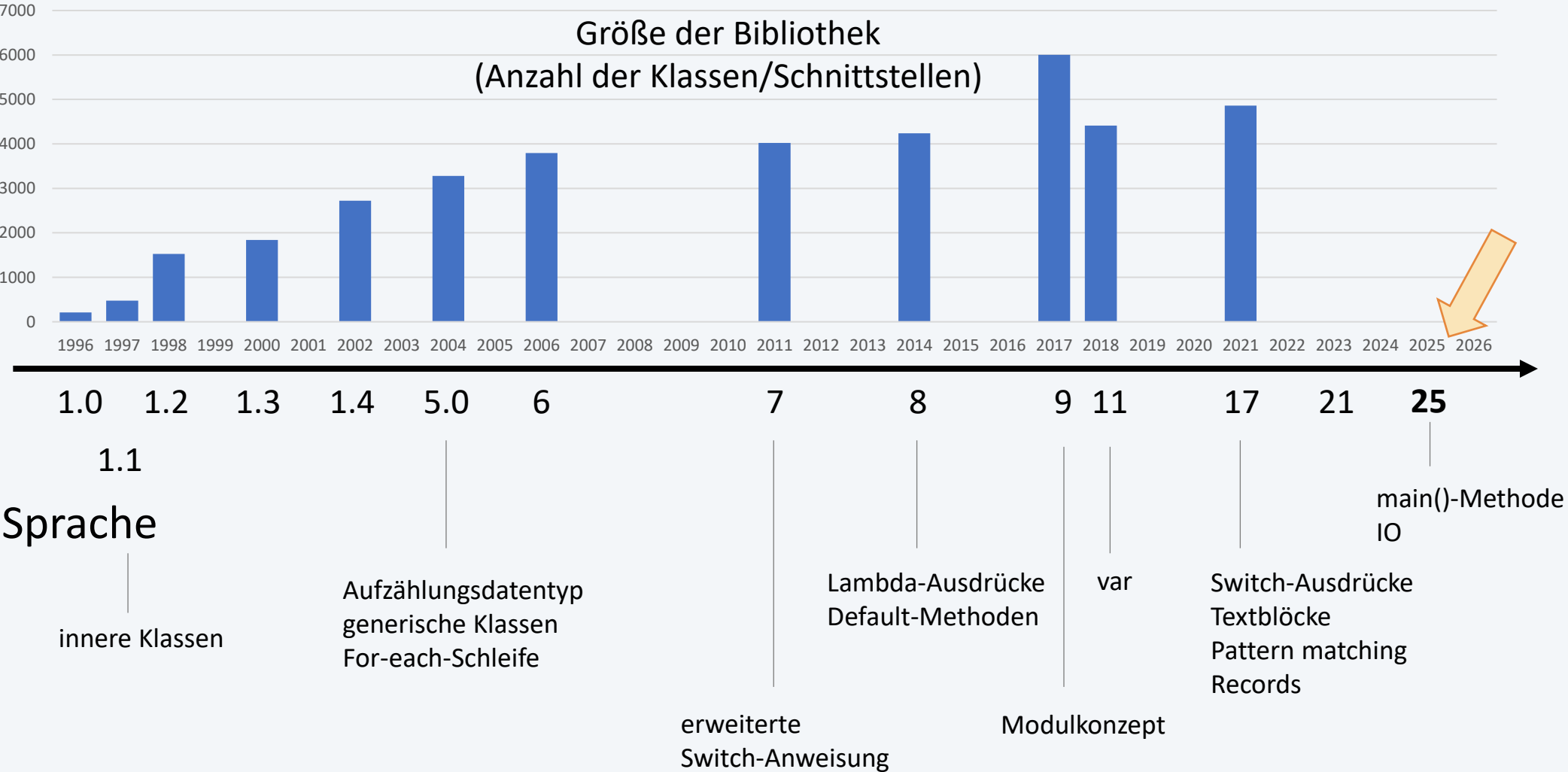


Java 25 Standard Edition (SE) (Sep. 2025)

- **Java SE** - die Java-Plattform für den Einsatz auf Desktops und Servern
- **Java Development Kit (JDK)** - die Software für Programmierer*innen, die Java-Programme schreiben
- **Java Runtime Environment (JRE)** - die Software zum Ausführen von Java-Programmen, ohne Entwicklungswerkzeuge
- **OpenJDK** - eine Open-Source-Implementierung von Java SE

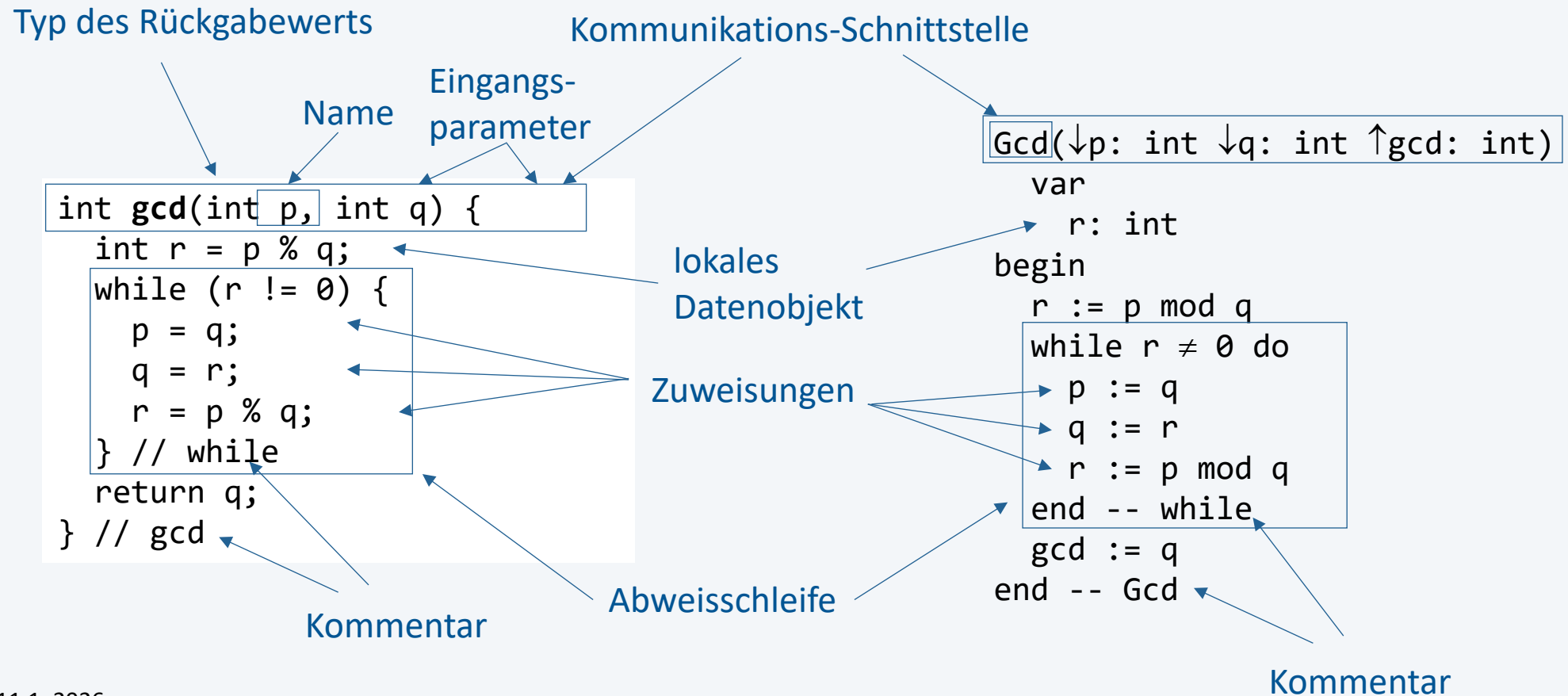
Einfach. Leicht zu erlernen und zu verwenden (gilt für frühe Versionen); Syntax ist einfacher als von Vorbild C++.	Architekturneutral. Der Compiler erzeugt einen architekturneutralen Maschinencode (Java-Bytecode).
Objektorientiert. Fokus liegt auf Objekten und Klassen; einfache und gute Form der OOP.	Portabel. Keine maschinen- oder compilerabhängige Aspekte; definierte Größe von Datentypen; int ist immer ein 32-Bit-Integer.
Verteilt. Bibliothek für Netzwerkschnittstellen wie HTTP, FTP u. RMI; Zugriff auf Dateien und Objekte über URLs so einfach wie auf Dateisystem.	Interpretiert. Virtuelle Maschine (JRE) führt Java-Bytecode aus und lädt erforderlichen Bytecode dynamisch nach.
Robust. Compiler findet viele Programmierfehler; starke Typisierung, Ausnahmebehandlung und automatische Speicherbereinigung.	Leistungsstark. Schnelle Ausführung durch Just-in-Time-Übersetzung in echten Maschinencode.
Sicher. Verhindert viele Angriffsmöglichkeiten (z.B. Speicherüberlauf, Dateizugriff ohne Berechtigung); Sandkastenmodell.	Leichtgewichtige Prozesse. Pseudo-parallele Ausführung von Anweisungen und Sprachkonstrukte zur Synchronisierung.
Dynamisch. Laden von Bytecode (Klassen) zur Laufzeit möglich; gute Unterstützung der Softwareevolution.	

Bibliothek



Algorithmen bestehen aus Datenobjekten und Anweisungen (Aktionen) zu deren Manipulation.

Beispiel: Algorithmus von Euklid



Klassenmethode `main()`

Die Java Virtual Machine (JVM) führt bei Programmstart die Klassenmethode `main()` aus (z.B. `java FirstProgram`).

Textdatei mit dem Namen `FirstProgram.java`

Klassenmethode `main()` („Hauptprogramm“)

```
void main() {  
    String s = "Hagenberg";  
    IO.println(s);  
    for (int i = 0; i < s.length(); i++) {  
        IO.print("*");  
    }  
}
```

```
PROGRAM FirstProgram;  
  VAR  
    s: STRING;  
    i: INTEGER;  
  BEGIN  
    s := 'Hagenberg';  
    WriteLn(s);  
    FOR i := 1 TO Length(s) DO  
      Write('*')  
    END.  
END.
```

Klassenmethode `main()`

Vor Java 25 musste man die Klasse, die die `main`-Klassenmethode enthält, ausdrücklich deklarieren. Das ist nicht mehr erforderlich.

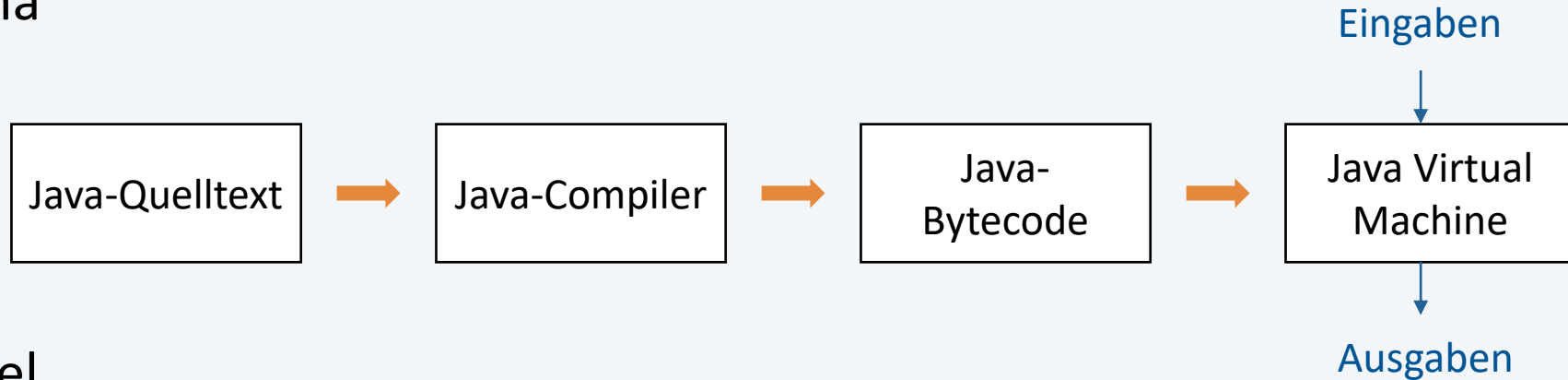
Früher war es außerdem erforderlich, die `main`-Klassenmethode wie folgt zu deklarieren:

```
public static void main(String[] args) { ... }
```

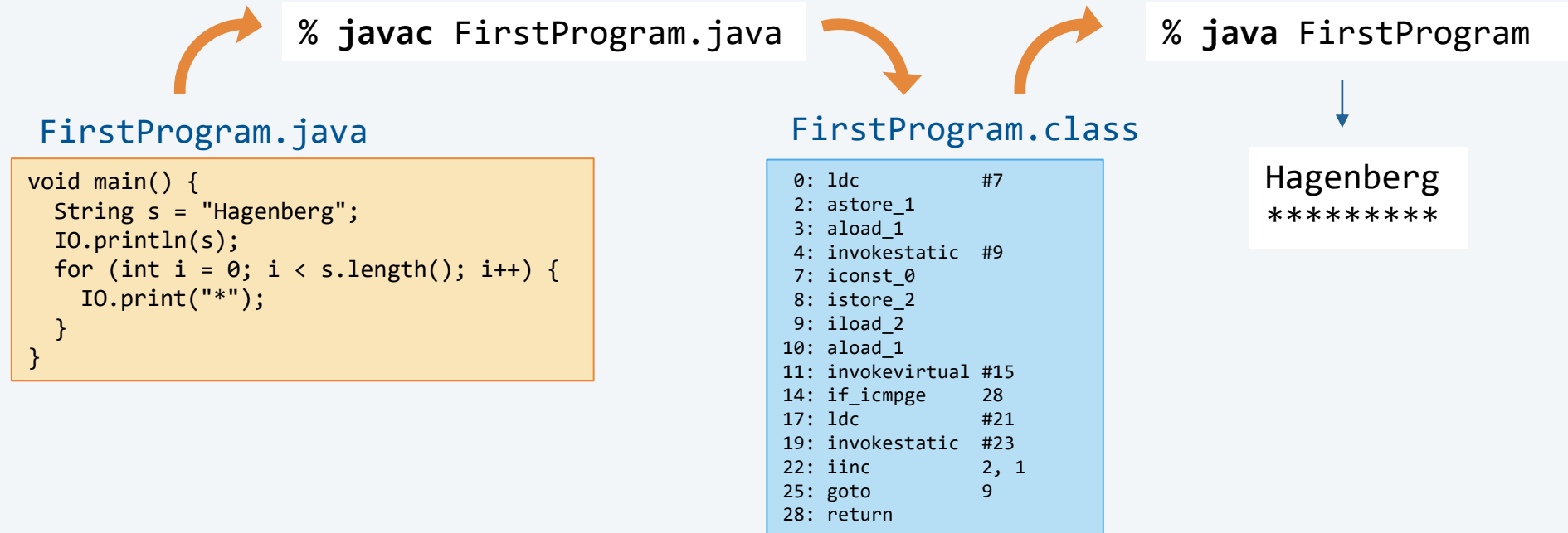
Mit Java 25 wurde Klasse `IO` eingeführt, um die Konsoleneingabe und -ausgabe zu vereinfachen (`IO.print` statt `System.out.print`).

```
public class FirstProgram {  
    public static void main(String[] args) {  
        String s = "Hagenberg";  
        System.out.println(s);  
        for (int i = 0; i < s.length(); i++) {  
            System.out.print("*");  
        }  
    }  
}
```

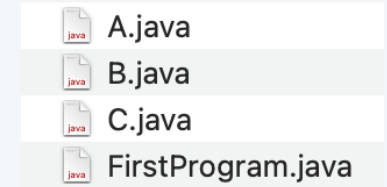

Schema



Beispiel



- ein Java-Programm besteht aus einer oder mehreren Klassen
- Klassen werden in Dateien organisiert
- jede Quelltext-Datei hat den Dateityp `.java`
- mindestens eine Klasse muss eine Klassenmethode `main` haben

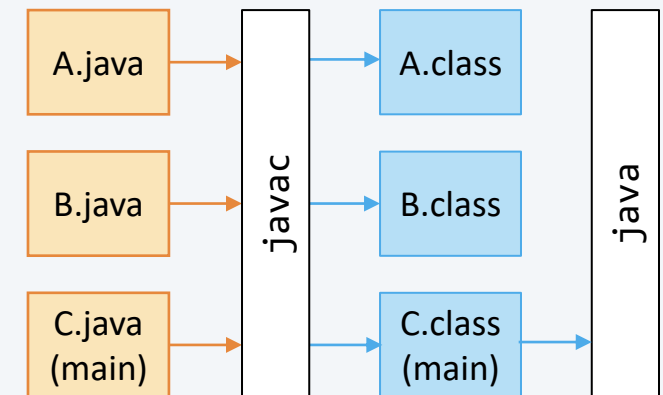


```
void main() { ... }
```

```
public static void main(String[] args) { ... }
```

 vor Java 25

- **Übersetzung:** der Compiler (`javac`) erzeugt aus jeder Quelltext-Datei eine (oder mehrere) Bytecode-Dateien (Dateityp `.class`)
- **Ausführung:** die Java Virtual Machine (`java`) wird mit dem Namen der Klasse aufgerufen, deren Klassenmethode `main` auszuführen ist, z. B. `java C`



Kommandozeilen-Argumente werden in einem Feld `args` übergeben

- Elemente sind `String`-Datenobjekte
- das erste Element steht an der Position 0
- die Anzahl der Elemente kann über `args.length` abgefragt werden

Beispiel

```
void main(String[] args) {  
    for (int i = args.length - 1; i >= 0; i--) {  
        IO.print(args[i]);  
        IO.print(" ");  
    }  
    IO.println();  
}
```

```
% javac Arguments.java
```

```
% java Arguments eins 2 drei  
drei 2 eins
```

```
PROGRAM Arguments;  
    VAR  
        i: INTEGER;  
BEGIN  
    FOR i := ParamCount DOWNTO 1 DO  
        Write(ParamStr(i), ' ');  
    Writeln;  
END.
```

Grundsymbole

■ Namen

i	args
s	println
gcd	System
main	FirstProgram

bezeichnen Variablen, Typen, ...

- bestehen aus Buchstaben, Ziffern und _
- beginnen mit Buchstaben oder _
- beliebig lang
- Groß-/Kleinschreibung signifikant

■ Schlüsselwörter

public	for
while	return

- dürfen nicht als Namen verwendet werden

■ Zahlen und Zeichen

42	'a'
2.78	'π'

- ganze Zahlen (dezimal oder hexadezimal)
- Kommazahlen

■ Zeichenketten

"Hagenberg"
"*"
"ab\"cd"

- beliebige Zeichenfolgen zwischen Hochkommas
- dürfen nicht über Zeilengrenzen gehen
- " in der Zeichenkette wird als \" geschrieben

Reservierte Schlüsselwörter

abstract	continue	for	new	switch
assert	default	goto	package	synchronized
boolean	do	if	private	this
break	double	implements	protected	throw
byte	else	import	public	throws
case	enum	instanceof	return	transient
catch	extends	int	short	try
char	final	interface	static	void
class	finally	long	strictfp	volatile
const	float	native	super	while
				—

Kontextabhängige Schlüsselwörter (können ggf. auch als Namen verwendet werden)

exports	open	provides	sealed	uses
module	opens	record	to	var
non-sealed	permits	requires	transitive	when
				with
				yield

true, false und null sind Literale

- traditioneller Kommentar: der gesamte Text von den Zeichen `/*` bis zu den Zeichen `*/` wird ignoriert (wie in C und C++)
- Zeilenkommentar: der gesamte Text ab den Zeichen `//` bis zum Ende der Zeile wird ignoriert (wie in C++)

```
/* Swap the pivot into its final position. */  
a[low] = a[lower]; a[lower] = pivot;
```

```
// Swap the pivot into its final position.  
a[low] = a[lower]; a[lower] = pivot;
```

```
/* Use an inexpensive approximation of the  
   golden ratio to select five sample elements  
   and determine pivots. */  
int step = (size >> 3) * 3 + 3;
```

```
// Use an inexpensive approximation of the  
// golden ratio to select five sample elements  
// and determine pivots.  
int step = (size >> 3) * 3 + 3;
```

- geschachtelte Kommentare sind nicht möglich

```
/* this comment /* // /* ends here: */
```

gültiger Kommentar

```
/* this comment /* // */ ends here: */
```

ungültiger Kommentar

↑
*/ an dieser Stelle beendet den Kommentar

Dokumentationskommentar: beginnt mit `/**` und endet mit `*/`

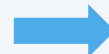
Generierung von HTML-Dokumentation mit dem Werkzeug `javadoc`

Gcd.java

```
/**
 * Computes the greatest common divisor (GCD) of two positive integers.
 *
 * @param p the first positive integer
 * @param q the second positive integer
 * @return the greatest common divisor of {@code p} and {@code q}
 */
public static int gcd(int p, int q) {
    int r = p % q;
    ...
}
```



`% javadoc Gcd.java`



Method Details

gcd

```
public static int gcd(int p,
                      int q)
```

Computes the greatest common divisor (GCD) of two positive integers.

Parameters:

p - the first positive integer

q - the second positive integer

Returns:

the greatest common divisor of p and q

Gcd.html

- Kommentare, Zeichen, Zeichenketten und Namen (z.B. Variablennamen) werden aus Unicode-Zeichen gebildet

```
// π is the ratio of the circumference ...  
IO.println('π');  
IO.println("π is the ...");  
IO.println(π);
```

Alternativ mit `\uxxxx`:

π	IO.println('\u03C0');
π is the ...	IO.println("\u03C0 is the ...");
3.141592	IO.println(\u03C0);

← π ist eine Variable (hier ohne Deklaration)

- der 16-Bit Unicode-Zeichensatz ist mit dem ASCII-Zeichensatz kompatibel
- ein Unicode-Zeichen kann durch eine Unicode-Escape-Sequenz festgelegt werden, z.B.: `\uxxxx`, wobei xxxx aus Hexadezimalzeichen besteht

```
for (char ch = '♔'; ch <= '♚'; ch++) {  
    IO.print(ch);  
}  
  
for (char ch = '\u2654'; ch <= '\u265F'; ch++) {  
    IO.print(ch);  
}
```

♔ ♚ ♜ ♝ ♞ ♟ ♠ ♡ ♢ ♣ ♤ ♥ ♦ ♧ ♨

♔ ♚ ♜ ♝ ♞ ♟ ♠ ♡ ♢ ♣ ♤ ♥ ♦ ♧ ♨

- Unterstützung der C-Zeichendarstellungen, wie `\n` (newline)



Bildquelle:

<https://de.wikipedia.org/wiki/Java-Technologie>

1 Einführung in Java

1.1 Java SE

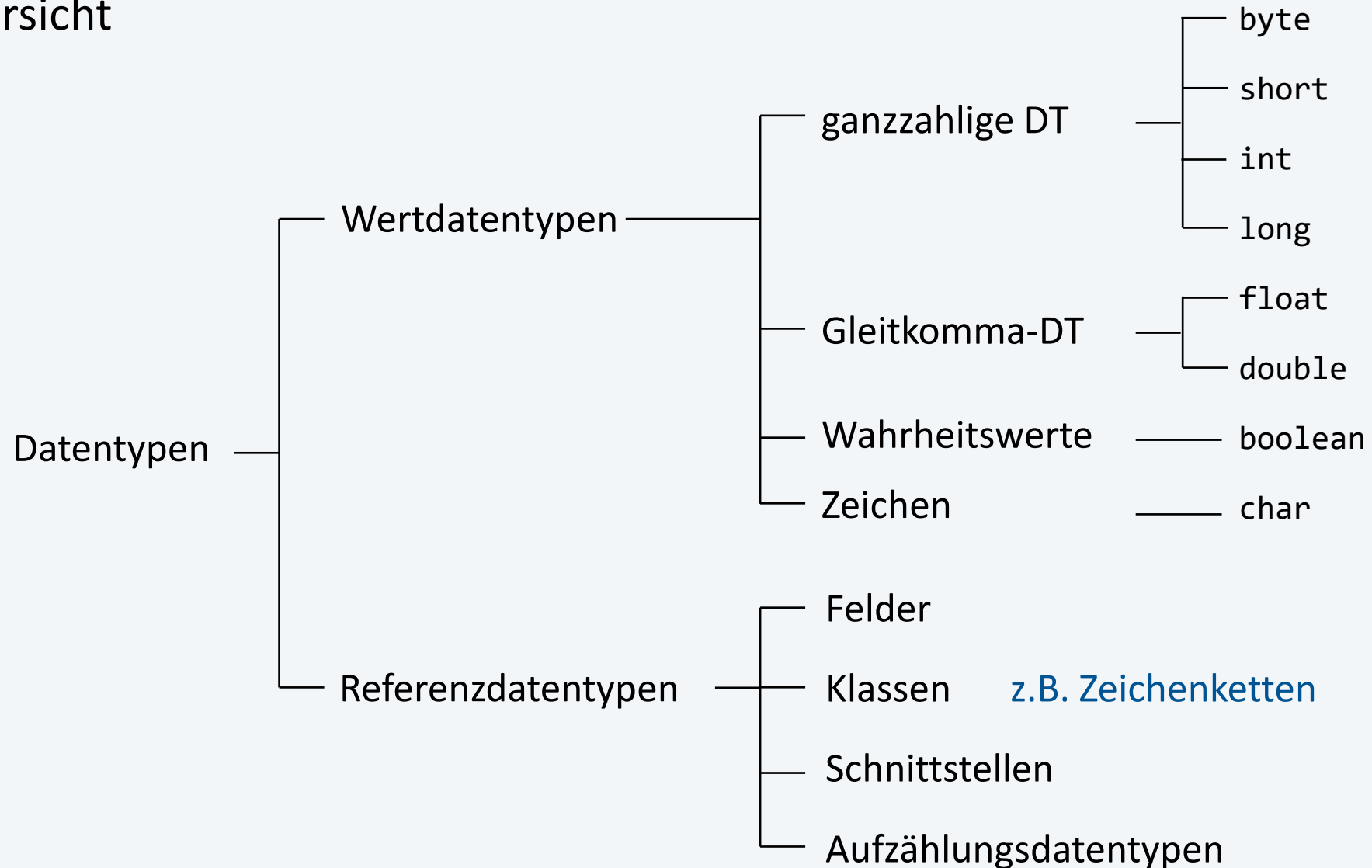
1.2 Typsystem

1.3 Anweisungen

1.4 Klassenmethoden

1.5 Felder

Übersicht



- jede Variable muss vor ihrer Verwendung deklariert werden

```
double salary;  
int vacationDays;  
long earthPopulation;  
boolean done;
```

```
int i, j, k;  
double αλφα;  
boolean 👑;
```

Unzulässiges Zeichen: "\u2654"

- Variablen können bei ihrer Deklaration initialisiert werden

```
double salary = 42000.0;  
int vacationDays = 30;  
long earthPopulation = 8261746187L;
```

```
int i = 1, j = 2, k = 3;  
double αλφα = 1.234;
```

- der Java-Compiler stellt sicher, dass lokale Variablen vor ihrer Verwendung initialisiert wurden

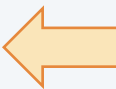
```
int vacationDays;  
IO.println(vacationDays);
```

^^^^^^^^^^^^

local variable `vacationDays` may not have been initialized

- Werte repräsentieren ganze Zahlen (alle mit Vorzeichen)

byte	8 Bit	$-2^7 .. 2^7 - 1$	-128 .. 127
short	16 Bit	$-2^{15} .. 2^{15} - 1$	-32 768 .. 32 767
int	32 Bit	$-2^{31} .. 2^{31} - 1$	-2 147 483 648 .. 2 147 483 647
long	64 Bit	$-2^{63} .. 2^{63} - 1$	-9 223 372 036 854 775 808 ..

 wir verwenden
in der Regel **int**

- Literale haben Typ `int` oder `long` (mit Suffix `l` oder `L`)

10	<code>int</code>	1_000_000	<code>int</code>
10L	<code>long</code>	8_261_746_187L	<code>long</code>

- Kleinste und größte Werte

Byte.MIN_VALUE	Byte.MAX_VALUE
Short.MIN_VALUE	Short.MAX_VALUE
Integer.MIN_VALUE	Integer.MAX_VALUE
Long.MIN_VALUE	Long.MAX_VALUE

```
int a = Integer.MIN_VALUE;
```

Integer.java (in Java-Bibliothek enthalten)

```
public class Integer {  
  
    ... final int MIN_VALUE = -2147483648;  
    ... final int MAX_VALUE = 2147483647;  
  
}
```

 **final ... Konstanten**

- Prompt an ChatGPT (3.5, April 2023)

Write a program PowersOfK that takes an integer command-line argument k and prints all the positive powers of k in the Java long data type.

Note: The constant Long.MAX_VALUE is the value of the largest integer in long.

- Antwort

```
public class PowersOfK {  
    public static void main(String[] args) {  
        int k = Integer.parseInt(args[0]);  
        long result = 1;  
        for (int i = 0; i <= 63; i++) {  
            result = result * k;  
            if (result > Long.MAX_VALUE) {  
                System.out.println("The value of " + k + " raised to the ...  
                break;  
            }  
            System.out.println(k + " raised to the power of " + (i + 1) + ...  
        }  
    }  
}
```

+	Addition
-	Subtraktion
*	Multiplikation
/	Division
%	Modulo

<<	Shift Left
>>	Shift Right
>>>	Shift Right ohne VZ
+	Vorzeichen
-	Vorzeichen

&	bitweises <i>and</i>
^	bitweises <i>xor</i>
	bitweises <i>or</i>

- Division (/ und %) durch 0 führt zu **Laufzeitfehler**
- bei arithmetischen Operationen ist **Überlauf** möglich (kein Laufzeitfehler!)
- Ergebnistyp ist int oder long (wenn mind. ein Operand long ist)

```
short s;  
int i;  
long n;  
char c;
```

```
i = 2 * i;  
i = i + s;  
i = 2 * c;
```

```
n = 2 * n;  
n = n + i;  
n = n + s;
```

```
i = 2 * n;  
s = 2 * s;  
s = s + s;
```

^

incompatible types: possible lossy
conversion from int to short

- Vergleichsoperatoren ==, !=, <, >, <=, >=

- Werte repräsentieren reelle Zahlen mit Vorzeichen (Standard IEEE 754)

float	32 Bit	$\pm 3.402 \times 10^{38}$	Float.MIN_VALUE .. Float.MAX_VALUE
double	64 Bit	$\pm 1.797 \times 10^{308}$	Double.MIN_VALUE .. Double.MAX_VALUE

- Literale haben Typ float (mit Suffix f oder F) oder double
- ein E oder e steht für einen dezimalen Exponenten

3.14F	float	1.729E3	double 1729
3.14	double	.125	double
3.14D	double		

- arithmetische Operationen können spezielle Werte ergeben

Double.POSITIVE_INFINITY	1.0 / 0.0;
Double.NEGATIVE_INFINITY	-1.0 / 0.0;
Double.NaN	0.0 / 0.0;

sowie Float.NaN etc.

NaN = Not a Number

Operatoren

+	Addition
-	Subtraktion
*	Multiplikation
/	Division
%	Modulo

kein Laufzeitfehler bei
Division durch 0

5.5 * 2.0	11.0
5.5 / 2.0	2.75
5.5 % 2.0	1.5

- der Ergebnistyp entspricht dem größeren der beiden Operandentypen (double oder float)

```
int i;  
float f;  
double d;
```

```
f = i * f;  
d = i * f;  
d = f * d;
```

```
f = f * d;
```

^

incompatible types: possible lossy
conversion from double to float

- zwei Werte true und false

Operatoren

Kurzschlussauswertung
(short circuit evaluation)

<code>a && b</code>	Konjunktion, wertet b nur aus, wenn der Wert a true ist
<code>a b</code>	Disjunktion, wertet b nur aus, wenn der Wert a false ist
<code>a & b</code>	Konjunktion, wertet beide Operanden aus
<code>a b</code>	Disjunktion, wertet beide Operanden aus
<code>a ^ b</code>	exklusives ODER
<code>!a</code>	Negation

- die Operatoren `<` und `>` sind nicht definiert!
- keine automatische Umwandlung in und von anderen Typen

- Lösung:

```
boolean b;  
int i;
```

```
b = (i != 0);
```

```
if (b) {  
    i = 1;  
} else {  
    i = 0;  
}
```

- Wert repräsentiert ein 16-Bit-Unicode-Zeichen (kompatibel mit ASCII)
- ein Unicode-Zeichen kann durch eine Unicode-Escape-Sequenz festgelegt werden, z.B.: `\uxxxx`, wobei `xxxx` aus Hexadezimalzeichen besteht

```
for (char ch = 'A'; ch <= 'Z'; ch++) {  
    IO.print(ch);  
}
```

ABCDEFGHIJKLMNOPQRSTUVWXYZ

```
for (char ch = '\u2654'; ch <= '\u265F'; ch++) {  
    IO.print(ch);  
}
```

♔ ♚ ♖ ♜ ♟ ♞ ♗ ♘ ♙ ♠ ♡ ♢ ♣

- Operator `++`

```
char ch = 'A'; // or '\u0041'  
ch++;  
IO.print(ch)
```

B

- Zeichen können automatisch nach `int` umgewandelt werden

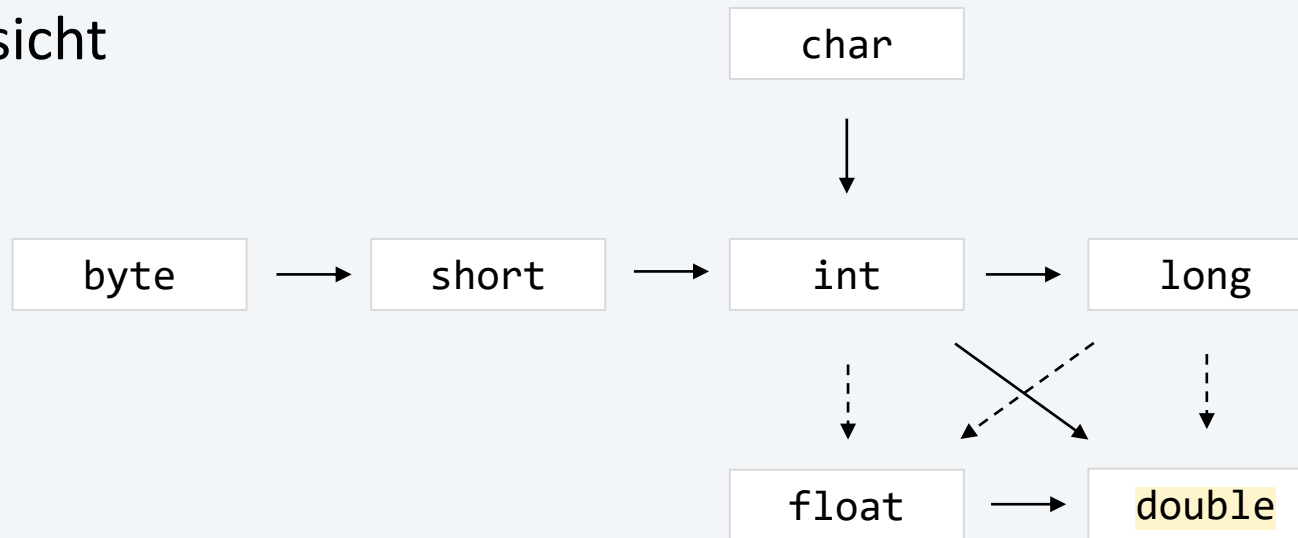
```
char ch = 'A';  
int i = ch;  
IO.print(i)
```

65

```
char ch = 65;  
IO.print(ch)
```

A

Übersicht



—> durchgezogenen Pfeile bezeichnen Umwandlungen ohne Informationsverlust

---> gestrichelte Pfeile bezeichnen Umwandlungen mit pot. Informationsverlust

Beispiel: Typumwandlung mit Informationsverlust

```
int i = 123456789;  
float f = i;  
IO.print(f);
```

1.2345679E8

Increment (++) und Decrement (--)

Postfix-Operatoren

<code>i++</code>	liefert Wert von <code>i</code> und erhöht <code>i</code> anschließend um 1
<code>i--</code>	liefert Wert von <code>i</code> und verringert <code>i</code> anschließend um 1

Prefix-Operatoren

<code>++i</code>	erhöht <code>i</code> um 1 und liefert anschließend den geänderten Wert von <code>i</code>
<code>--i</code>	verringert <code>i</code> um 1 und liefert anschließend den geänderten Wert von <code>i</code>

Beispiele

```
i = 2;  
j = 3 * i++;
```

`i = 3, j = 6`

```
i = 2;  
j = 3 * ++i;
```

`i = 3, j = 9`

- kann als Anweisung verwendet werden (guter Stil!)

```
i++;
```

 statt `i = i + 1;`

Aufgabe: warum ist Ausdruck `3 * ++i++` nicht möglich?

- Zuweisung erfolgt durch Zuweisungsoperator (=) nach Auswertung des Ausdrucks auf der rechten Seite

```
i = 42
```

```
int i, j, k;
```

- wird in der Regel als Anweisung verwendet

```
i = i + 1;
```

```
k = j;  
j = i;  
i = k;
```

- binäre Operatoren lassen sich mit Zuweisung kombinieren

+=	i += j	i = i + j
-=	i -= j	i = i - j
...	...	...

<<=	i <<= j	i = i << j
&=	i &= j	i = i & j
...	...	...

- möglich, aber schlechter Stil:

```
i = j = k = 3;
```

```
i += j += k;
```

Operanden von Operatoren werden i.d.R. von links nach rechts ausgewertet
zwei Ausnahmen mit Reihenfolge von rechts nach links:

- Ausdrücke mit unären Operatoren, z.B. $\sim x$ bedeutet $\sim(x)$
- Zuweisungen

```
i = j = k = 3;  
i += j += k;
```

Vorrang

Operators	Associativity
[] , () (method call)	Left to right
! ~ ++ -- + (unary) - (unary) () (cast) new	Right to left
* / %	Left to right
+ -	Left to right
<< >> >>>	Left to right
< <= > >= instanceof	Left to right
== !=	Left to right
&	Left to right
^	Left to right
	Left to right
&&	Left to right
	Left to right
?:	Right to left
= += -= *= /= %= &= = ^= <<= >>= >>>=	Right to left

Klassenmethoden in der Klasse Math (Auszug)

<code>double Math.abs(double a)</code>	Absolutwert
<code>double Math.min(double a, double b)</code>	Minimum
<code>double Math.max(double a, double b)</code>	Maximum

auch für int, long
und float

<code>double Math.sin(double a)</code>	Sinus (Radianten)
<code>double Math.cos(double a)</code>	Cosinus (Radianten)
<code>double Math.tan(double a)</code>	Tangens (Radianten)

<code>double Math.exp(double a)</code>	e^a
<code>double Math.log(double a)</code>	$\ln a$
<code>double Math.pow(double a, double b)</code>	a^b

<code>long Math.round(double a)</code>	nähere ganze Zahl
<code>double Math.sqrt(double a)</code>	Quadratwurzel
<code>double Math.random()</code>	Zufallszahl (0..1(

auch
`int round(float)`

- Beispiele: Felder, Klassen und Schnittstellen
- Java kennt nur dynamische Objekte und Felder (am Heap)
- Felder und Objekte sind nur indirekt über ihre **Referenzen** zugänglich
- alle benutzerdefinierten DT sind Referenz-DT

Referenzen

- keine Zeiger
- keine Typumwandlung von Referenzdatentypen auf primitive Datentypen und umgekehrt
- keine Ermittlung von Adressen einer Variablen
- keine Ermittlung der Größe von primitiven Datentypen oder von Referenzdatentypen
- keine Zeigerarithmetik

Klassen und Objekte

```
class ListNode {  
    ListNode next;  
    int data;  
}
```

- Variablen eines Referenzdatentyps enthalten eine Referenz

```
ListNode list;
```

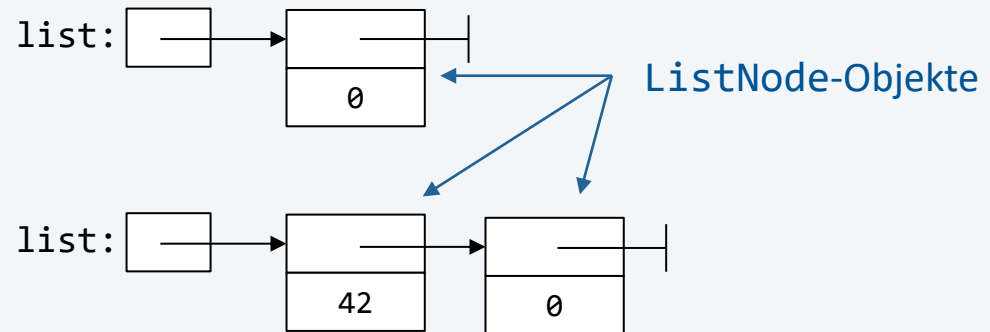
list:  Referenz auf ListNode-Objekt

- der Operator new erzeugt ein Objekt (allokiert Speicherplatz)

```
list = new ListNode();
```

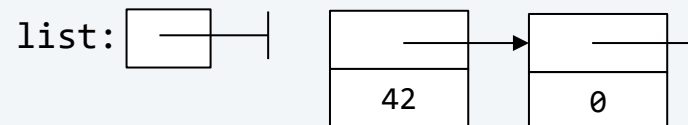
- Zugriff auf Datenkomponenten

```
list.data = 42;  
list.next = new ListNode();
```



- vordefinierter Wert null bedeutet *ungültige Referenz*
- null kann jeder Variablen eines Referenzdatentyps zugewiesen werden (Ausnahme von der starken Typisierung)

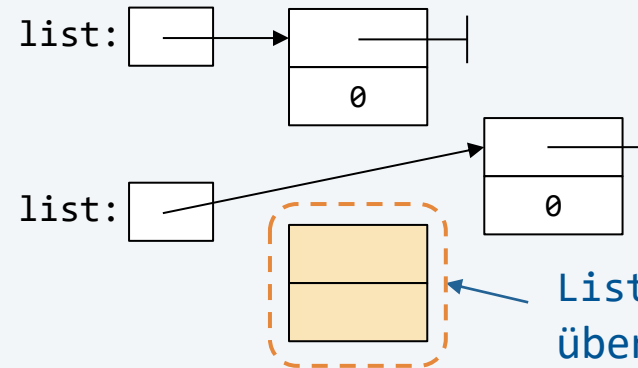
```
list = null;
```



- es gibt kein Gegenstück zum Operator new, also kein delete
- Objekte, die nicht mehr erreichbar sind, können automatisch freigegeben werden

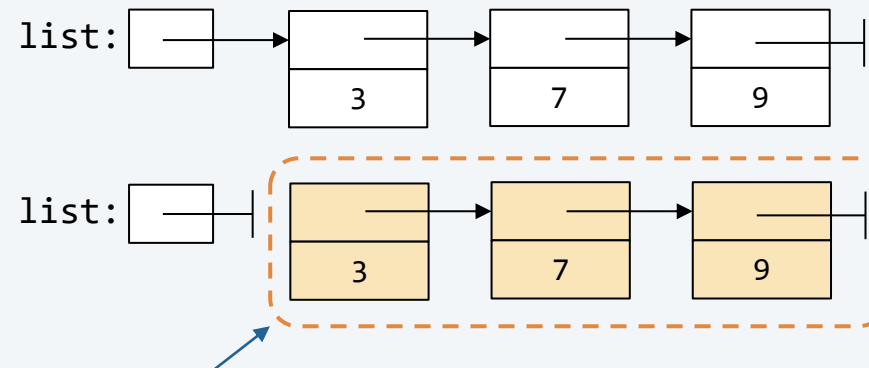
```
ListNode list = new ListNode();
```

```
list = new ListNode();
```



```
ListNode list = new ListNode();  
...
```

```
list = null;
```



ListNode-Objekte sind nicht mehr erreichbar

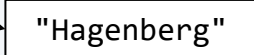
- Zeichenketten sind Objekte der Klasse String

```
String s;
```

s:  Referenz auf String-Objekt

- Objekte der Klasse String repräsentieren Folgen von Unicode-Zeichen
- ein String-Objekt hat einen unveränderlichen Wert
- String-Literale sind Referenzen auf Objekte der Klasse String (Compiler erzeugt automatisch String-Objekte)

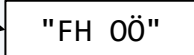
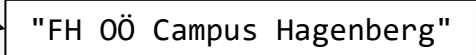
```
s = "Hagenberg";
```

s:   String-Objekt

- der Operator + erzeugt implizit ein neues String-Objekt

```
String fh = "FH OÖ";  
String fhch = fh + " Campus " + s;
```

 String-Literal

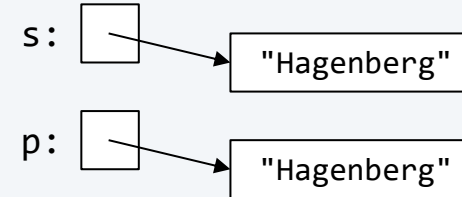
fh:  
fhch:  

Vergleich von Zeichenketten

```
s == p    false
```

```
s.equals(p)  true
```

String s, p;



Methoden der Klasse String (Auszug)

<code>int i = s.length();</code>	9	liefert die Anzahl der Zeichen in s
<code>char c = s.charAt(2);</code>	'g'	liefert Zeichen mit Index 2
<code>int i = s.indexOf("berg");</code>	5	liefert Index mit Übereinstimmung "berg"
<code>p = s.substring(5);</code>	"berg"	liefert Teilstring ab Index 5
<code>p = s.substring(2, 5);</code>	"gen"	liefert Teilstring ab Index 2 bis Index 5 (exklusiv)
<code>p = "FH OÖ ".concat(s);</code>	"FH..."	entspricht <code>p = "FH OÖ " + s;</code>

- Klasse `StringBuilder` bietet veränderbare Zeichenketten

```
StringBuilder s = new StringBuilder();  
s.append("Hagenberg");  
s.append("Mühlkreis");  
s.insert(9, " am ");  
s.setCharAt(10, 'i');  
String r = s.toString();
```

```
""  
"Hagenberg"  
"HagenbergMühlkreis"  
"Hagenberg am Mühlkreis"  
"Hagenberg im Mühlkreis"  
"Hagenberg im Mühlkreis"
```

- Methoden der Klasse `StringBuilder` (Auszug)

auch für `int`, `long`,...

<code>StringBuilder append(String str)</code>	hängt <code>str</code> an Zeichenfolge an
<code>StringBuilder insert(int offset, String str)</code>	fügt <code>str</code> an Position <code>offset</code> ein
<code>StringBuilder replace(int start, int end, String str)</code>	ersetzt Teil-String durch <code>str</code>
<code>StringBuilder delete(int start, int end)</code>	entfernt Zeichen im Bereich <code>start..end</code>
<code>StringBuilder setCharAt(int index, char ch)</code>	setzt Zeichen <code>ch</code> an Position <code>index</code>
<code>StringBuilder reverse()</code>	ersetzt Zeichenkette mit umgekehrter F.
<code>String toString()</code>	gibt <code>String</code> -Objekt zurück

- der Operator + ist sinnvoll, wenn Zeichenketten aus nur wenigen Teilketten gebildet werden
- darüber hinaus ist die Verwendung der Klasse StringBuilder effizienter

Vergleich

```
int n = 100000;  
StringBuilder s = new StringBuilder();  
for (int i = 0; i < n; i++) {  
    s.append("*");  
}  
String r = s.toString();
```

```
int n = 100000;  
String s = "";  
for (int i = 0; i < n; i++) {  
    s = s + "*";  
}  
String r = s;
```

n	StringBuilder	String
1000	41 ms	35 ms
100000	44 ms	273 ms
200000	46 ms	851 ms
300000	50 ms	1819 ms

- werden durch Aufzählung ihrer Werte deklariert

```
enum ChessPiece {  
    QUEEN, KING, ROOK, BISHOP, KNIGHT, PAWN  
}
```

- Variablen und Zuweisung

```
ChessPiece p = ChessPiece.QUEEN;
```

```
p = ChessPiece.KNIGHT;
```

```
p = null;
```

- Aufzählungswerte müssen mit Typnamen qualifiziert werden
- Aufzählungswerte sind Objekte
- Aufzählungsdatentyp ist Referenzdatentyp

- Operatoren und Methoden

```
p == ChessPiece.KNIGHT
```

```
p != ChessPiece.KNIGHT
```

```
p.ordinal()
```

```
p.compareTo(ChessPiece.KNIGHT)
```

- kein Vergleich mit <, <= etc.
- Methode `ordinal` liefert Ordinalwert (Start mit 0)
- Methode `compareTo` bildet Differenz der Ord.
 - < 0 ... p < ChessPiece.KNIGHT
 - 0 ... p = ChessPiece.KNIGHT
 - > 0 ... p > ChessPiece.KNIGHT

- zwischen String und primitiven Typen

String s = String.valueOf(1234);	String	"1234"
String s = String.valueOf(1234.56789);	String	"1234.56789"
String s = String.valueOf(false);	String	"false"
int i = Integer.parseInt("1234");	int	1234
double d = Double.parseDouble("1234.56789");	double	1234.56789
boolean b = Boolean.parseBoolean("true");	boolean	true

Beispiel

```
public class LeapYear {  
    public static void main(String[] args) {  
        int year = Integer.parseInt(args[0]);  
        boolean isLeapYear;  
        isLeapYear = (year % 4 == 0);  
        isLeapYear = isLeapYear && (year % 100 != 0);  
        isLeapYear = isLeapYear || (year % 400 == 0);  
        IO.println(String.valueOf(isLeapYear));  
    }  
}
```

```
% java LeapYear 2024  
true
```

```
% java LeapYear 2025  
false
```


- zwischen Zahlenwerten

Beispiele

```
double d = 19.94;  
int i = (int)d; 19
```

```
double d = 1.1 / 10.0;  
float f = (float)d; 0.110000000000000001  
0.11
```

```
int n = 45;  
double r = Math.random();  
int i = (int)(r * n); liefert double-Zufallszahl im Bereich (0..1(  
i hat Wert zwischen 0 und n - 1
```

```
int j = (int)r * n; j ist immer 0
```

- die Ausgabe von Variablen / Ausdrücken erfolgt mittels Klassenmethoden `print` und `println` der Klasse `IO`

Typ `byte`, `short`, `int`, `long`, `float`, `double`, `boolean`, `char`, `String`

<code>IO.print(x);</code>	gibt den Wert von <code>x</code> aus
<code>IO.println(x);</code>	gibt den Wert von <code>x</code> aus und beginnt eine neue Zeile
<code>IO.println();</code>	beginnt eine neue Zeile

- formatierte Ausgabe

```
int i = 42;
double x = 10.0 / 3.0;
boolean b = true;
```

```
IO.println("i = %d".formatted(i));
```

```
i = 42
```

```
IO.println("x = %f, b = %b".formatted(x, b));
```

```
x = 3.333333, b = true
```

<code>%n</code>	beginnt neue Zeile
<code>%d</code>	<code>byte</code> , <code>short</code> , <code>int</code> , <code>long</code>
<code>%f</code>	<code>float</code> , <code>double</code>
<code>%b</code>	<code>boolean</code>
<code>%s</code>	<code>String</code>

Mit Java 25 wurde Klasse `IO` eingeführt, um die Konsoleneingabe und -ausgabe zu vereinfachen (`IO.print` statt `System.out.print`).

- die Ausgabe von Variablen / Ausdrücken erfolgt über das vordefinierte Objekt `System.out` mittels Methoden `print`, `println`, und `format`

<code>System.out.print(x);</code>	gibt den Wert von x aus
<code>System.out.println(x);</code>	gibt den Wert von x aus und beginnt eine neue Zeile

Typ `byte`, `short`, `int`, `long`, `float`, `double`,
`boolean`, `char`, `String`

```
int i = 123456; int j = 12; double x = Math.PI;
```

```
"i = %d".formatted(i);
```

```
i = 123456
```

```
"i = %10d, j = %10d".formatted(i, j);
```

```
i =      123456, j =      12
```

```
"i = %-10d, j = %-10d".formatted(i, j);
```

```
i = 123456      , j = 12
```

```
"x = %.1f%n".formatted(x);
```

```
x = 3.1
```

```
"x = %8.2f%n".formatted(x);
```

```
x =      3.14
```

Beispiel

```
for (int year = 1; year <= 10; year++) {  
    double dogYearOld = year * 7.0;  
    double dogYearNew = 16.0 * Math.log(year) + 31.0;  
    IO.println("%2d %7.2f %7.2f".formatted(year, dogYearOld, dogYearNew));  
}
```

```
1      7.00    31.00
```

```
2     14.00    42.09
```

```
3     21.00    48.58
```

```
...
```

```
10    70.00    67.84
```

- die Eingabe von Zeichenketten erfolgt mittels Klassenmethoden `readln` der Klasse `IO`

<code>String line = IO.readln();</code>	liest eine einzelne Textzeile
<code>IO.println("Text = ");</code>	schreibt Prompt Wert <code>a</code> und liest eine einzelne Textzeile

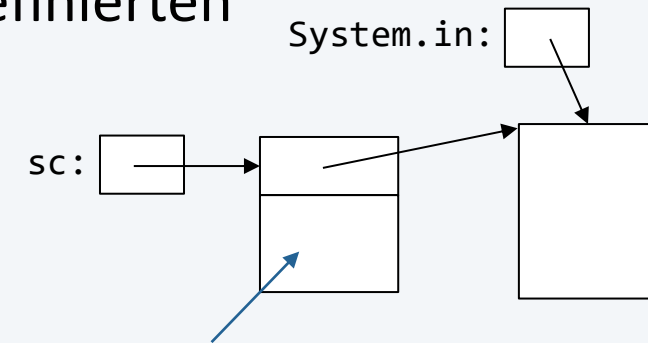
- die Eingabe von Zahlen etc. erfolgt über Eingabe einer Zeichenkette und anschließender Konvertierung

```
String line = IO.readln("Zahl a = ");
int a = Integer.parseInt(line);

IO.println("%d³ = %d".formatted(a, a*a*a));
```

- zum Einlesen von Werten verschiedener Datentypen kann ein Scanner-Objekt verwendet werden, das mit dem vordefinierten Objekt `System.in` verbunden ist

```
Scanner sc = new Scanner(System.in);  
String line = sc.nextLine();  
int i = sc.nextInt();  
double x = sc.nextDouble();  
sc.close();
```



Objekt der Klasse Scanner

- Methoden der Klasse Scanner (Auszug)

<code>int i = sc.nextInt();</code>	liest eine ganze Zahl
<code>byte b = sc.nextByte();</code>	liest eine ganze Zahl
<code>long n = sc.nextLong();</code>	liest eine ganze Zahl
<code>float f = sc.nextFloat();</code>	liest eine Gleitkommazahl
<code>double d = sc.nextDouble();</code>	liest eine Gleitkommazahl
<code>String s = sc.nextLine();</code>	liest nächste Zeile



Bildquelle:

<https://de.wikipedia.org/wiki/Java-Technologie>

1 Einführung in Java

1.1 Java SE

1.2 Typsystem

1.3 Anweisungen

1.4 Klassenmethoden

1.5 Felder

- die leere Anweisung besteht nur aus ;
- Ausdrücke können als Anweisungen verwendet werden, indem sie mit ; abgeschlossen werden

```
a = a + 1;  
a += 1;  
a++;
```

Blöcke

- geschweifte Klammern { und }
- die Blockanweisung ist syntaktisch äquivalent zu einer einzelnen Anweisung
- die innerhalb eines Blocks deklarierten lokalen Variablen haben nur lokale Gültigkeit

```
{  
    int a = 4;  
    int b = 5;  
    {  
        int t = a;  
        a = b;  
        b = t;  
    }  
}
```



```
if (Bedingung) Anweisung  
if (Bedingung) Anweisung else Anweisung
```

- Bedingung muss ein boolescher Ausdruck sein

```
if (x < 0)  
    x = -x;
```

```
if (a > b)  
    max = a;  
else  
    max = b;
```

```
if (a > b)  
    if (a > c)  
        d = a;  
    else  
        d = b;
```

else gehört immer zum unmittelbar davorstehenden if (dangling else)

- mehrere Anweisungen müssen durch { ... } geklammert werden

```
if (a > b) {  
    max = a;  
    min = b;  
} else {  
    max = b;  
    min = a;  
}
```

```
if (a > b) {  
    max = a;  
} else {  
    max = b;  
}
```

```
if (a > b) {  
    int t = a;  
    a = b;  
    b = a;  
}
```

Variable t hat lokale Gültigkeit

$(\text{Bedingung}) \text{ ? Ausdruck}_1 : \text{Ausdruck}_2$

- Bedingung muss ein boolescher Ausdruck sein
- Ausdruck_1 : Das Ergebnis dieses Ausdrucks wird zurückgegeben, wenn Bedingung den Wert `true` liefert
- Ausdruck_2 : Das Ergebnis dieses Ausdrucks wird zurückgegeben, wenn Bedingung den Wert `false` liefert
- Ausdruck_1 und Ausdruck_2 sollen vom gleichen Typ sein

```
max = (a > b) ? a : b;
```

```
value = 2 * ((a > b) ? a : b) + 3;
```

schlechter Stil

```
int year = 2025;  
String century = (year < 2000) ? "20th" : "21th";
```

```
int v = (p != null) ? p.m() : 0;
```

```
if (a > b) {  
    max = a;  
} else {  
    max = b;  
}
```

```
assert Bedingung  
assert Bedingung : Ausdruck
```

- Bedingung muss ein boolescher Ausdruck sein
- wenn Bedingung `false` liefert, wird ein Laufzeitfehler gemeldet
- Ausdruck kann z.B. für eine Fehlermeldung verwendet werden
- bei Programmstart müssen Assertionen (Zusicherungen) explizit aktiviert werden (Option `-ea`)

```
assert n != null : "Node must not be null";  
n.data = 3;
```

```
java -enableassertions Assertions  
Exception in thread "main" java.lang.AssertionError: Node must not be null  
    at ch3.Assertions.main(Assertions.java:7)
```



```
java -ea Assertions
```

switch-Anweisung (klassische Form)

- Ausdruck liefert einen ganzzahligen Wert, Zeichen oder Zeichenkette (String)
- abhängig vom Ergebnis des Ausdrucks wird zur entsprechenden case-Marke verzweigt
- jede case-Marke wird optional mit einer break-Anweisung beendet
- existiert keine case-Marke für das Ergebnis des Ausdrucks wird zur default-Marke verzweigt
- default-Marke ist optional
- jede case-Marke kann beliebig viele Anweisungen enthalten

```
switch (Ausdruck) {  
    case Ausdruck1:  
        Anweisungen1  
        break;  
    case Ausdruck2:  
        Anweisungen2  
        break;  
    case ...  
  
    default:  
        Anweisungend  
}
```

switch-Anweisung (klassische Form)

```
enum Weekday {  
    MON, TUE, WED, THU, FRI, SAT, SUN  
}
```

Beispiele

```
int i;
```

```
switch (i) {  
    case 1:  
        e();  
    case 2:  
        f();  
        break;  
    case 3:  
    case 4:  
        g();  
        h();  
        break;  
    default:  
        x();  
}
```

```
String direction;
```

```
switch (direction) {  
    case "north":  
        y++;  
        break;  
    case "south":  
        y--;  
        break;  
    case "east":  
        x++;  
        break;  
    case "west":  
        x--;  
        break;  
    default:  
        x = 0; y = 0;  
}
```

```
Weekday day;
```

```
switch (day) {  
    case MON:  
        lessons = 6; break;  
    case TUE:  
        lessons = 5; break;  
    case WED:  
        lessons = 7; break;  
    case THU:  
        lessons = 6; break;  
    case FRI:  
        lessons = 4; break;  
    case SAT:  
    case SUN:  
        lessons = 0; break;  
}
```

auch möglich:

```
case SAT, SUN:  
    lessons = 0; break;
```

while (Bedingung) Anweisung

- solange die Bedingung erfüllt ist, wird Anweisung ausgeführt
- die Bedingung muss ein boolescher Ausdruck sein
- mehrere Anweisungen müssen durch { ... } geklammert werden

Beispiele

```
int sum = 0;
int i = 1;
while (i <= n) {
    sum += i;
    i++;
}
```

```
int i = 10;
while (i > 0) {
    ...
    i = calculate();
    ...
}
```

do Anweisung **while** (Bedingung)

- Anweisung wird einmal ausgeführt **und** solange die Bedingung erfüllt ist, wird Anweisung immer wieder ausgeführt
- die Bedingung muss ein boolescher Ausdruck sein
- mehrere Anweisungen müssen durch { ... } geklammert werden

```
int sum = 0;
int i = 1;
do {
    sum += i;
    i++;
} while (i <= n)
```

for (Ausdruck₁; Ausdruck₂; Ausdruck₃) Anweisung

- Ausdruck₁ (optional) dient der Initialisierung; lokale Zählervariablen können deklariert werden
- Ausdruck₂ (optional) dient dem Test der Abbruchbedingung (wird immer wieder ausgewertet!)
- Ausdruck₃ (optional) legt fest, um welchen Wert die Zählervariable bei jedem Schleifendurchlauf verändert wird
- mehrere Anweisungen im Schleifenrumpf müssen durch { ... } geklammert werden

guter Stil

```
int sum = 0;
for (int i = 1; i <= n; i++) {
    sum += i;
}
```

schlechter Stil

```
for (;;) {
    IO.println("*");
}
```


Entwickeln Sie ein Java-Programm, das ein quadratisches Muster der Größe $n \times n$ ausgibt.

```
void main() {
    String line = IO.readLine("n = ");
    int n = Integer.parseInt(line);
    if (n >= 3) {
        for (int row = 0; row < n; row++) {
            for (int col = 0; col < n; col++) {
                if (row == 0 || row == n - 1 || col == 0 || col == n - 1) {
                    IO.print("#");
                } else {
                    char ch = ((row + col) % 2 == 0) ? 'X' : 'O';
                    IO.print(ch);
                }
                IO.print(' ');
            }
            IO.println();
        }
    }
}
```

```
# # # # #
# X O X O #
# O X O X #
# X O X O #
# O X O X #
# # # # #
```

- mit der break-Anweisung wird aus der innersten Wiederholungsanweisung oder switch-Anweisung gesprungen
- die break-Anweisung kann um ein Sprungziel ergänzt werden
- das Sprungziel muss sich in einem umschließenden Block befinden

Beispiel: Vorzeitiger Abbruch in Schleifen (siehe Kapitel 4)

```
while (2 * k <= n) {  
    int j = 2 * k;  
    if (j < n && less(j, j + 1)) {  
        j++;  
    }  
    if (!less(k, j)) {  
        break;  Position gefunden  
    }  
    swap(k, j);  
    k = j;  
}
```

- mit der `continue`-Anweisung wird an das Ende der innersten Wiederholungsanweisung gesprungen
- die `continue`-Anweisung kann um ein Sprungziel ergänzt werden
- das Sprungziel muss sich in einem umschließenden Block befinden (in diesem Fall wird an das Ende der Schleife gesprungen, zu der das Sprungziel gehört)

```
int goal = 100;
int sum = 0;
while (sum < goal) {
    String line = IO.readLine("n = ");
    int n = Integer.parseInt(line);
    if (n < 0) {
        IO.println("Negative number, try again");
        continue;
    }
    sum += n;
}
```

wird nicht ausgeführt, wenn $n < 0$



Bildquelle:

<https://de.wikipedia.org/wiki/Java-Technologie>

1 Einführung in Java

1.1 Java SE

1.2 Typsystem

1.3 Anweisungen

1.4 Klassenmethoden

1.5 Felder

- Objekte bestehen aus Datenkomponenten und Methoden
- Methoden werden auf Objekte aufgerufen
- Klassenmethoden sind „Algorithmen“, die unabhängig von Objekten aufgerufen werden

Klassenmethode parseInt

```
int i = Integer.parseInt("1234");
```

Klasse

Name der
Klassenmethode

Methode length

```
String s = "Hello";  
int i = s.length();
```

Objekt

Name der
Methode

Beispiel: Aufruf der Klassenmethode für Berechnung GGT(gcd)

```
int a = 378;  
int b = 216;  
int c = Num.gcd(a, b);
```

Klasse

Name der
Klassenmethode

Aufbau von Klassenmethoden (`static`)

- Schnittstelle besteht aus Typ des Rückgabewerts, Name der Methode und Liste von Eingangsparametern (in Java gibt es nur `call-by-value`)
- Klassenmethoden mit Rückgabewert müssen mit `return`-Anweisung beendet werden
- für Klassenmethoden ohne Rückgabewert wird `void` angegeben

Beispiel: Klassenmethode für Berechnung des GGT(gcd)

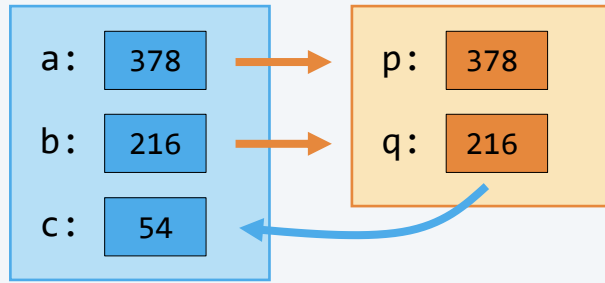
Typ des Rückgabewerts Name Eingangsparameter
(jeweils Typ und Name)

```
public class Num {  
    static int gcd(int p, int q) {  
        int r = p % q;  
        while (r != 0) {  
            p = q; q = r; r = p % q;  
        }  
        return q;  
    }  
}
```

Schlüsselwort für
Klassenmethode

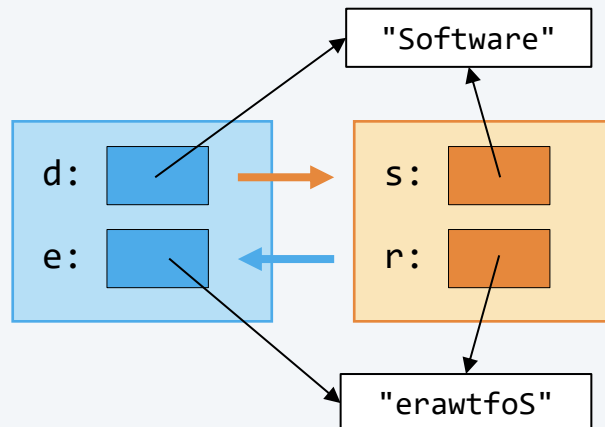
Parameterübergabe (call-by-value)

```
int a = 378;  
int b = 216;  
int c = Num.gcd(a, b);
```



```
public class Num {  
    static int gcd(int p, int q) {  
        ...  
        return q;  
    }  
}
```

```
String d = "Software";  
String e = Str.reverse(d);
```



```
public class Str {  
    static String reverse(String s) {  
        String r = "";  
        for (int i = 0; i < s.length(); i++)  
            r = s.charAt(i) + r;  
        return r;  
    }  
}
```

- mehrere Klassenmethoden können den gleichen Namen haben
- jedoch unterschiedliche Parameterlisten (Anzahl / Typen von Parametern)

```
public class Num {  
    static int max(int a, int b) {  
        return (a > b) ? a : b;  
    }  
    static int max(int a, int b, int c) {  
        return max(max(a, b), max(a, c));  
    }  
    static double max(double a, double b) {  
        return (a > b) ? a : b;  
    }  
}
```

①
②
③

- beim Aufruf wählt der Java-Compiler die richtige Methode aus

```
int x, y, z; double u, v;  
int m = Num.max(x, y);  
int n = Num.max(x, y, z);  
double o = Num.max(u, v);
```

(int, int) → ①
(int, int, int) → ②
(double, double) → ③



Bildquelle:

<https://de.wikipedia.org/wiki/Java-Technologie>

1 Einführung in Java

1.1 Java SE

1.2 Typsystem

1.3 Anweisungen

1.4 Klassenmethoden

1.5 Felder

- Felder sind Objekte (dynamische Datenobjekte)
- in einem Feld können Werte primitiver Datentypen oder von Referenzdatentypen gespeichert werden

Deklaration

- bei der Variablendeklaration wird nur der Elementtyp angegeben

```
int[] a;
```

ebenfalls möglich:

```
int a[];
```

a: 

Referenz auf
int-Feld

Erzeugen eines Feldes

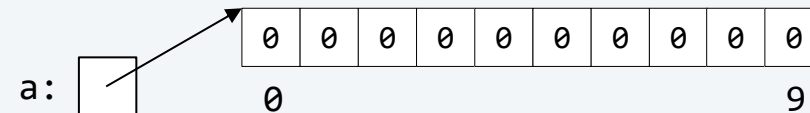
- mit dem Operator new, dem Elementtyp und Angabe der Feldgröße (Typ int)

```
int[] a = new int[10];
```

```
int n = ...;
```

```
int[] a = new int[n * n];
```

```
int[] a = new int[0];
```

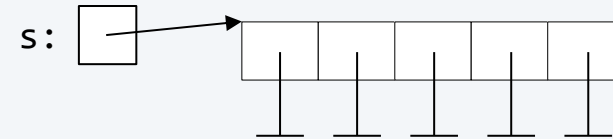


beliebiger Ausdruck mit Typ int

Feld der Länge 0 ist möglich

- Elemente werden mit Default-Wert des Elementtyps initialisiert
(0, 0.0, false, null)

```
String[] s = new String[5];
```

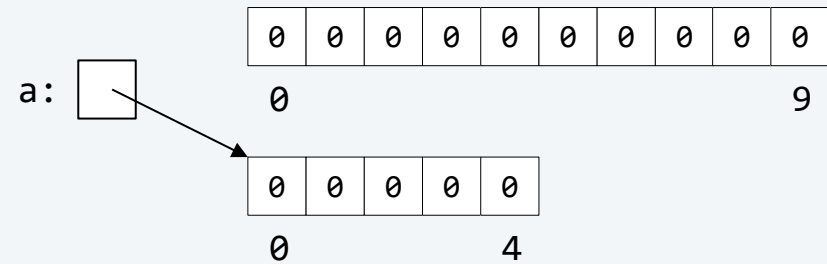


- mehrmalige Zuweisung möglich

```
int[] a;
```

```
a = new int[10];
```

```
a = new int[5];
```



Erzeugen eines Feldes durch Angabe von Initialwerten

```
int[] fib = new int[] {0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55};
```

```
int[] fib = {0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55};
```

- Zugriff auf Feldelemente erfolgt mittels Indizierung [...] und einem ganzzahligen Ausdruck (int)
- die Länge eines Feldes wird über die Datenkomponente `length` abgefragt
- das erste Element hat den Index 0
- Zugriff auf Feldelemente immer mit Laufzeitprüfung und ggf. -fehler

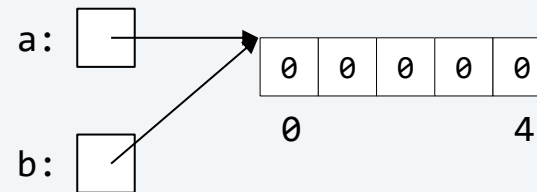
```
String[] s = new String[5];  
s[0] = "Hagenberg";  
s[1] = " ";  
s[2] = "im";  
s[3] = " ";  
s[4] = "Mühlkreis";
```

```
for (int i = 0; i < s.length; i++) {  
    IO.print(s[i]);  
}
```

Hagenberg im Mühlkreis

Wertzuweisung bei Feldern

```
int[] a, b;  
a = new int[5];  
b = a;
```



```
double[] a = new double[...];
```

Initialisierung mit Zufallswerten

```
for (int i = 0; i < a.length; i++) {  
    a[i] = Math.random();  
}
```

Kopieren der Elemente von a nach b

```
double[] b = new double[a.length];  
for (int i = 0; i < a.length; i++) {  
    b[i] = a[i];  
}
```

Ausgabe der Elemente

```
for (int i = 0; i < a.length; i++) {  
    IO.println(a[i]);  
}
```

```
0.8670168816848519  
0.9246841888363735  
0.39067873280839027  
...
```

Summe der Werte im Feld

```
double sum = 0.0;  
for (int i = 0; i < a.length; i++) {  
    sum += a[i];  
}
```

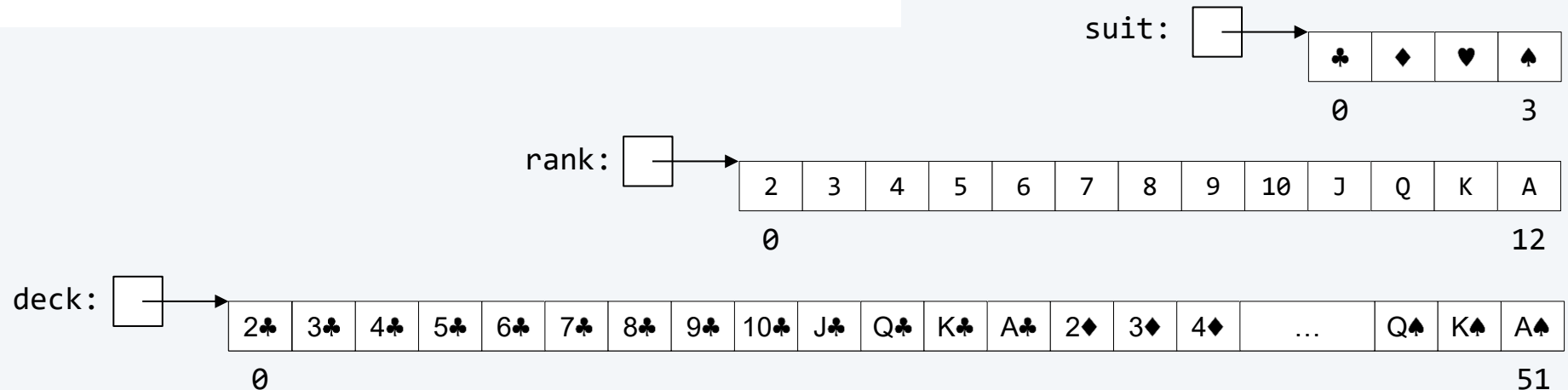
Beispiel: Datenstruktur für Kartendeck mit 4×13 Karten

verwende drei Felder

```
String[] rank = {"2", "3", "4", "5", "6", "7", "8", "9", "10", "J", "Q", "K", "A"};  
String[] suit = {"♣", "♦", "♥", "♠"};  
String[] deck = new String[rank.length * suit.length];
```

initialisiere Datenstruktur für Kartendeck

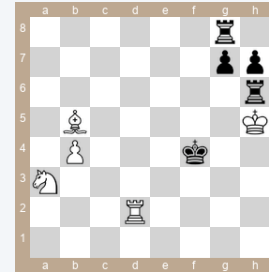
```
for (int i = 0; i < suit.length; i++) {  
    for (int j = 0; j < rank.length; j++) {  
        deck[13 * i + j] = rank[j] + suit[i];  
    }  
}
```



Mehrdimensionale Felder sind tatsächlich Felder von Feldern

```
ChessPiece[][] chessboard = new ChessPiece[8][8];
```

```
double[][] matrix = new double[3][4];
```

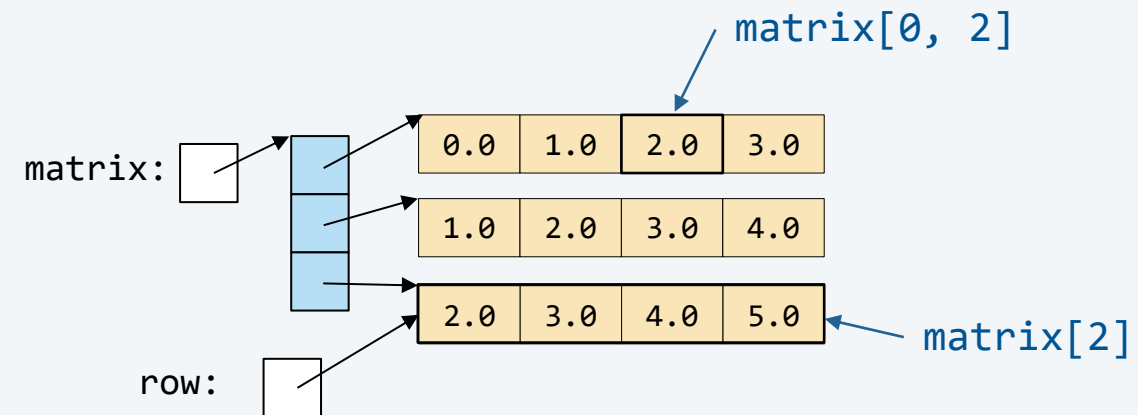


$$\begin{pmatrix} 1 & 2 & 9 & 1 \\ 4 & 8 & 3 & 5 \\ 3 & 7 & 1 & 6 \end{pmatrix}$$

Zugriff auf Feldelemente

```
matrix[0][2] = 1.3;
```

```
for (int i = 0; i < 3; i++) {  
    for (int j = 0; j < 4; j++) {  
        matrix[i][j] = i + j;  
    }  
}
```



Zugriff auf inneres Feld

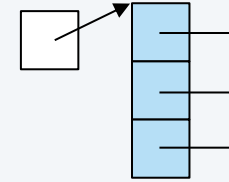
```
double[] row = matrix[2];
```

Mehrdimensionale Felder können unterschiedlich lange Teilfelder enthalten

```
double[][] jagged = new double[3][];
```

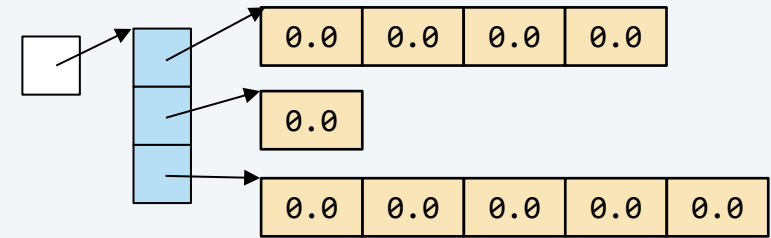
Größe für erste Dimension
muss angegeben werden

jagged:



```
jagged[0] = new double[4];  
jagged[1] = new double[1];  
jagged[2] = new double[5];
```

jagged:



```
jagged.length      3  
jagged[0].length   4
```

```
for (int i = 0; i < jagged.length; i++) {  
    for (int j = 0; j < jagged[i].length; j++) {  
        IO.print(jagged[i][j] + " ");  
    }  
    IO.println();  
}
```

```
0.0 0.0 0.0 0.0  
0.0  
0.0 0.0 0.0 0.0 0.0
```


for (VarDekl : Ausdruck) Anweisung

- VarDekl deklariert eine lokale Variable mit dem Elementtyp des Feldes und einem Namen
- Ausdruck liefert (Referenz auf) Feld
- Anweisung wird mit jedem Wert (Kopie des Elements) aus Feld ausgeführt

```
double[] a = new double[...];
```

Erweiterte for-Anweisung

```
for (double e : a) {  
    IO.println(e);  
}
```

```
double sum = 0.0;  
for (double e : a) {  
    sum += e;  
}
```

Zählschleife

```
for (int i = 0; i < a.length; i++) {  
    System.out.println(a[i]);  
}
```

```
double sum = 0.0;  
for (int i = 0; i < a.length; i++) {  
    sum += a[i];  
}
```

in der Klasse Arrays sind Standardalgorithmen für Felder implementiert

```
int[] a = new int[11];  
Arrays.fill(a, 5);  
IO.println(Arrays.toString(a));
```

[5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5]



Methoden der Klasse Arrays (Auszug)

<code>String s = Arrays.toString(a);</code>	liefert Zeichenkette mit String-Repr. der Elemente
<code>Arrays.sort(a);</code>	sortiert Elemente mit Quicksort
<code>int i = Arrays.binarySearch(a, 2);</code>	sucht Element mit Wert 2 und liefert Index
<code>Arrays.fill(a, 5);</code>	befüllt alle Elemente mit Wert 5

Klassenmethode mit variabler Anzahl von Parametern

- Parameter (rest) wird vom Compiler auf ein Feld abgebildet

```
static int max(int first, int... rest) {  
    int max = first;  
    for (int e : rest) {  
        if (e > max) {  
            max = e;  
        }  
    }  
    return max;  
}
```

0 oder mehrere int-Parameter

```
for (int i = 0; i < rest.length; i++) {  
    if (rest[i] > max) {  
        max = rest[i];  
    }  
}
```

Verwendung

```
int m, x, y, z;  
m = Num.max(x);  
m = Num.max(x, y);  
m = Num.max(x, y, z);  
m = Num.max(x, y, z, 100);
```

variable Anzahl von Parametern
(hier vom Typ int)



Bildquelle:

<https://de.wikipedia.org/wiki/Java-Technologie>

1 Einführung in Java

1.1 Java SE

1.2 Typsystem

1.3 Anweisungen

1.4 Klassenmethoden

1.5 Felder